

L13: Transaction and Cursor

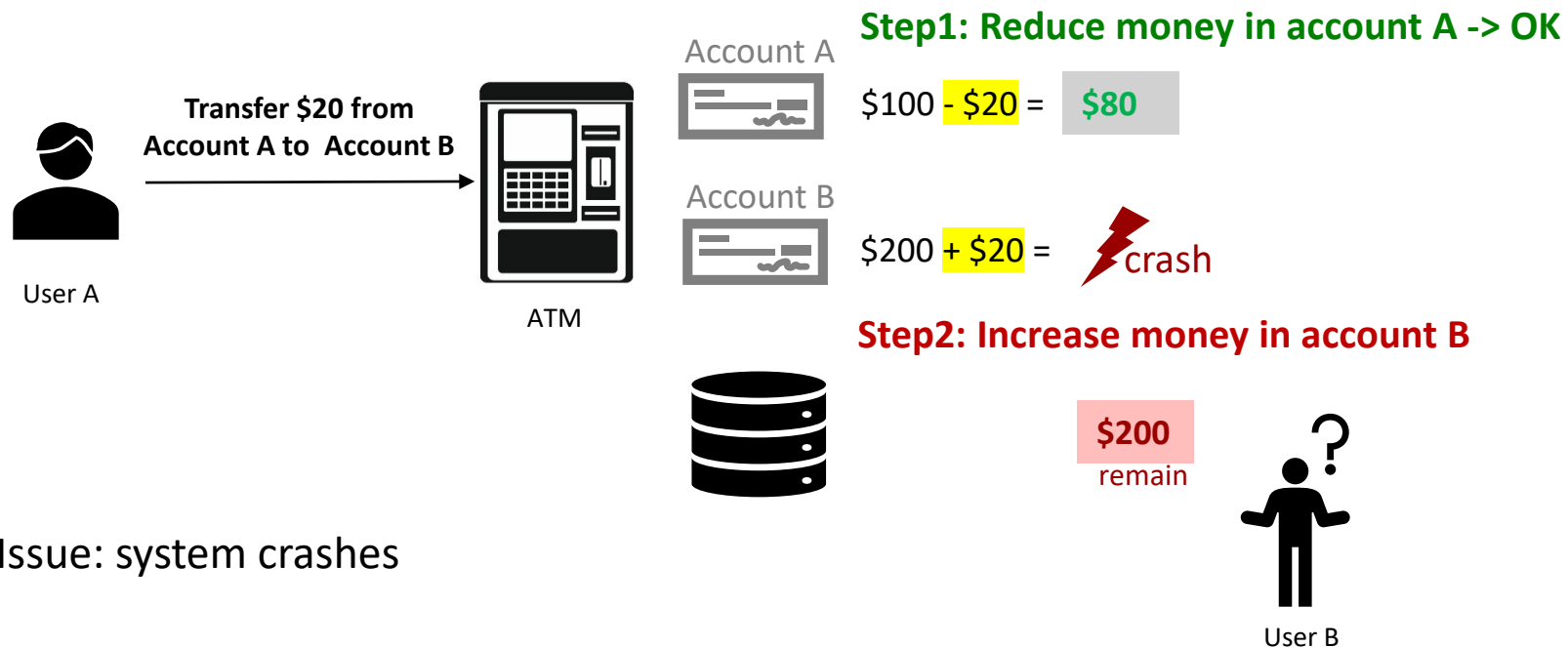
RECAP from Lab #12

- Trigger

```
DELIMITER //  
  
CREATE TRIGGER update_items_trigger  
AFTER UPDATE  
ON items  
FOR EACH ROW  
BEGIN  
    INSERT INTO item_changes (item_id, change_type)  
    VALUES (NEW.id, 'UPDATE');  
END;  
//  
  
DELIMITER ;
```

13.1 Transaction

Transaction is used to maintain a system's integrity and consistent state, by **ensuring** that interdependent operations on the system are either **all completed successfully**, or **all canceled successfully**.



Transaction Log

Keeps track of all transactions that update the database

DBMS uses the information stored in a log for:

- Recovery requirement triggered by a ROLLBACK statement
- A program's abnormal termination
- A system failure

Transaction Step

A COMMIT statement is reached, in which all changes are permanently recorded within the database.

A ROLLBACK statement is reached, in which all changes are aborted and the database is rolled back to its previous consistent state.

Transaction in MySQL

By default, when you execute an SQL statement, MySQL automatically wraps it in a transaction and commits the transaction automatically.

To instruct MySQL **to not** start a transaction implicitly and commit the changes automatically, you set the value of the `autocommit` variable to `0` or `OFF`

Syntax

```
SET autocommit = 0;
```

Create Transaction

Syntax

```
START TRANSACTION;  
    statement_1;  
    statement_2;  
    statement_3;  
{COMMIT | ROLLBACK};
```

- Transactions are usually used in Stored Procedures.
- COMMIT and ROLLBACK statements are applied using IF conditions.

Current Session

id	username	email
NULL	NULL	NULL
*		

```
START TRANSACTION;  
  
INSERT INTO users (id, username)  
VALUES (1, 'john');  
  
UPDATE users  
SET email = 'john.doe@example.com'  
WHERE id = 1;
```

id	username	email
1	john	john.doe@example.com
NULL	NULL	NULL

```
COMMIT;
```

Other Session

id	username	email
NULL	NULL	NULL
*		

id	username	email
1	john	john.doe@example.com
NULL	NULL	NULL

Current Session

id	username	email
1	john	john.doe@example.com
NULL	NULL	NULL

```
START TRANSACTION;  
  
UPDATE users  
SET email = 'abc.efg@example.com'  
WHERE id = 1;
```

id	username	email
1	john	abc.efg@example.com
NULL	NULL	NULL

```
ROLLBACK;
```

id	username	email
1	john	john.doe@example.com
NULL	NULL	NULL

Other Session

id	username	email
1	john	john.doe@example.com
NULL	NULL	NULL

id	username	email
1	john	john.doe@example.com
NULL	NULL	NULL

13.2 Lock / Unlock Table

A lock is a flag associated with a table. MySQL allows a client session to explicitly acquire a table lock to prevent other sessions from accessing the same table during a specific period.

Syntax

```
LOCK TABLES table_name {READ | WRITE};  
  
UNLOCK TABLES;
```

Syntax: Check the locked table

```
SHOW OPEN TABLES WHERE In_Use > 0 ;
```

Mode	Current Session	Other Session
READ	r	r
WRITE	r/w	-

13.3 Deadlock situation

A deadlock is a situation that occurs when two or more database transactions are blocked, waiting for each other to release locks on resources that the other transaction needs to proceed.

Example

process A Locks Table X

process B Locks Table Y

then

process A waits for Table Y and
process B waits for Table X } they wait forever!!

Deadlock

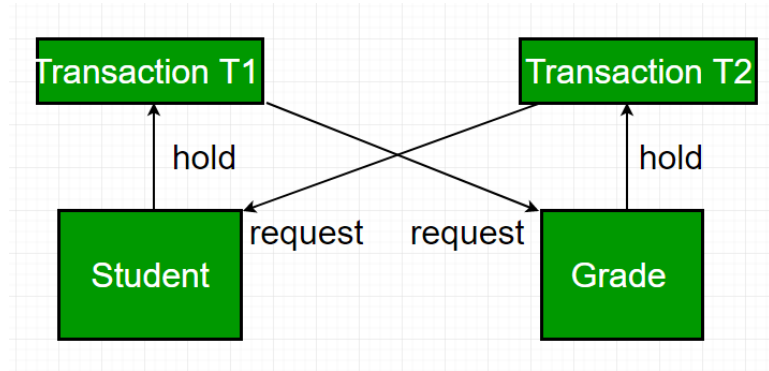
Example (without COMMIT)

```
--run at 1st instance
USE Transactions;
BEGIN;
SELECT acc_balance FROM accounts WHERE acc_id =1 FOR UPDATE;
```

```
-- run at 2nd instance
USE Transactions;
BEGIN;
SELECT acc_balance FROM accounts WHERE acc_id =2 FOR UPDATE;
```

```
-- run at 1st instance
SELECT acc_balance FROM accounts WHERE acc_id =2 FOR UPDATE;
```

```
-- run at 2nd instance
SELECT acc_balance FROM accounts WHERE acc_id =1 FOR UPDATE;
```



54 02:07:58 SELECT acc_balance FROM accounts WHERE acc_id =2 LIMIT 0, 1000 FOR UPDATE Error Code: 1213. Deadlock found when trying to get lock; try restarting transaction

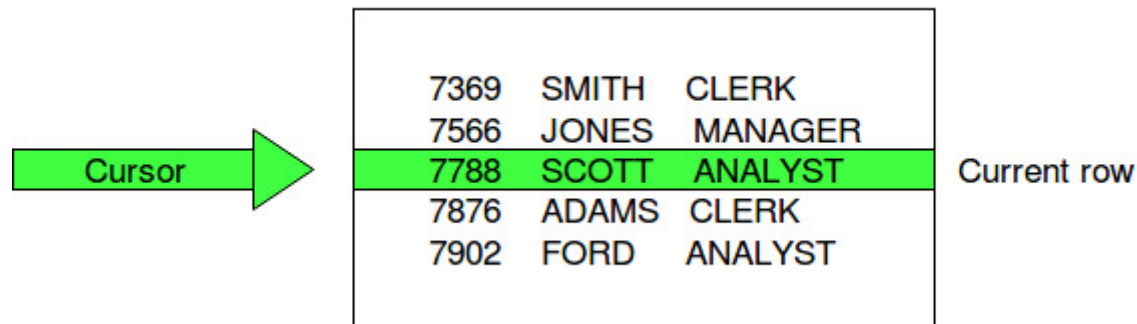
Error Code: 1213. Deadlock found when trying to get lock; try restarting transaction

13.4 Cursor

A Cursor is a pointer to a context area that stores the result set of a query.

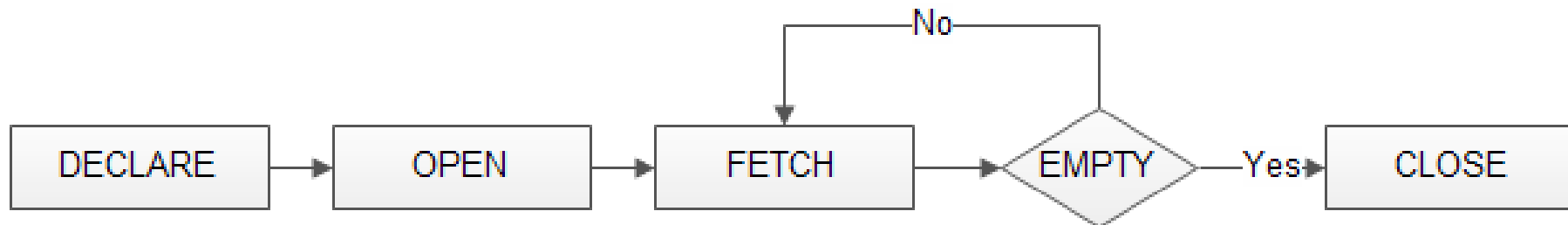
The cursor is used to **retrieve data one row at a time** from the results set, unlike other SQL commands that operate on all rows at once.

The data that is stored in the Cursor is called the Active Data Set.



Cursor Actions

1. **Declare Cursor:** A cursor is declared by defining the SQL statement that returns a result set.
2. **Open:** A Cursor is opened and populated by executing the SQL statement defined by the cursor.
3. **Fetch:** When the cursor is opened, rows can be fetched from the cursor one by one or in a block to perform data manipulation.
4. **Close:** After data manipulation, close the cursor explicitly.
5. **Deallocate:** Finally, delete the cursor definition and release all the system resources associated with the cursor.



Cursor Actions

Syntax

```
-- declare a cursor
DECLARE cursor_name CURSOR FOR
SELECT column1, column2
FROM your_table
WHERE your_condition;

-- open the cursor
OPEN cursor_name;

-- process the data
FETCH cursor_name INTO variable1, variable2;

-- close the cursor
CLOSE cursor_name;
```

Example

```
DELIMITER $$
CREATE PROCEDURE create_email_list (
    INOUT email_list TEXT
)
BEGIN
    DECLARE done BOOL DEFAULT false;
    DECLARE email_address VARCHAR(100) DEFAULT "";
    DECLARE cur CURSOR FOR SELECT email FROM employees;
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = true;

    OPEN cur;

    SET email_list = '';
    process_email: LOOP
        FETCH cur INTO email_address;
        IF done = true THEN
            LEAVE process_email;
        END IF;
        SET email_list = CONCAT(email_address, ";", email_list);
    END LOOP;

    CLOSE cur;

END$$
DELIMITER ;
```

```
CALL create_email_list(@email_list);
SELECT @email_list
```



@email_list
mgerard@classicmodelcars.com;ykato@classicmodelcars.com;mnishi@classicmodelcars.com

13.5 Export Database with Built Objects

